ODINGARD SECURITY — ENGINEERING SPEC V1.0

OdinForge AEV: The Parallel Architecture

Three transformative capabilities — parallel specialized agents, code-to-runtime context, and auto-remediation — architected for the real codebase.

300+	10-50x	6–8 wk
CONCURRENT MICRO-AGENTS	COVERAGE INCREASE	FULL BUILD TIMELINE

EXECUTIVE SUMMARY

OdinForge's current Phase 1 architecture runs a single Active Exploit Engine against each target followed by three sequential AI pipeline calls. This is fast and functional, but structurally narrow. A 30-endpoint target gets one engine pass and three evaluations. Aikido fans out 100+ specialized agents in parallel, each with a single focused hypothesis.

This document specifies how to close that gap — and in two areas, surpass it.

The three capabilities, in priority order:

Capability 1: Parallel Specialized Agents — Fan out micro-agents per endpoint × per vulnerability class. A Juice Shop scan hitting 30 endpoints × 10 vuln types dispatches 300 concurrent agents. Each agent is stateless, scoped, and disposable. This turns a 30-second surface-level scan into a 5-minute deep breach campaign with 10-50x the finding density. This is pure engineering work — the ParallelAgentDispatcher already exists at `server/services/parallel-agent-dispatcher.ts` with concurrency controls. The gap is wiring it into the breach chain per-phase rather than only as a standalone evaluation tool.

Capability 2: Code-to-Runtime Context — The Go agent (`odinforge-agent/internal/collector/`) already collects local open ports, running processes, network interfaces, and service metadata from every enrolled host. That telemetry is sent to the queue but never read back into the breach chain planner. Adding a `RuntimeContextBroker` that pulls this telemetry by asset ID and injects it as a structured context block into each phase's LLM prompt gives OdinForge host-level visibility without source code access. "The agent on this host sees port 5432 listening locally and a process named `postgres` with PID 4231" is materially better context than black-box HTTP probing alone.

Capability 3: Auto-Remediation Loop — The `POST /api/aev/evaluations/:id/verify-fix` endpoint already exists. The gap is the "generate fix" half: a `RemediationEngine` that takes a confirmed finding, generates a prioritized fix (code patch suggestion, config change, or WAF rule), stores it, and re-runs validation to confirm closure. This closes the AEV loop from *find* → *fix* → *verify*.

CAPABILITY 1: PARALLEL SPECIALIZED AGENTS

Current State

File: `server/services/breach-orchestrator.ts` — `executeApplicationCompromise()`

```
for (const assetId of chain.assetIds) {
  // Phase 1A: One ActiveExploitEngine pass
  activeExploitResult = await runActiveExploitEngine(exploitTarget, ...)

  // Phase 1B: Three sequential AI pipeline calls
  for (const exposureType of ["app_logic", "cve", "api_sequence_abuse"]) {
    const result = await runAgentOrchestrator(assetId, exposureType, ...)
  }
}
```

This is `N_assets × (1 engine + 3 sequential AI calls)` . For a single target with 30 endpoints and 10 vuln classes, coverage is approximately 3% of the theoretical attack surface.

File: `server/services/parallel-agent-dispatcher.ts` — Already built, max 5 concurrent agents. Used only in standalone evaluation mode, never called from the breach chain.

Target Architecture

```
Phase 1A: Active Exploit Engine (existing — crawl + harvest endpoints)
   ↓ produces: DiscoveredEndpoint[]
Phase 1B: MicroAgentOrchestrator
   → for each endpoint × vuln_class → spawn MicroAgent
   → run up to 50 concurrent agents (semaphore-gated)
   → each agent: focused HTTP attack + LLM validation
   → findings merge into breach phase result
Phase 1C: Subdomain Agent Dispatch (already built today)
   → alive subdomains → per-subdomain MicroAgentOrchestrator
```

New File: `server/services/aev/micro-agent-orchestrator.ts`

The MicroAgentOrchestrator receives the crawl results from Phase 1A (which produces `DiscoveredEndpoint[]`) and fans out one agent per `(endpoint, vulnClass)` pair.

```
export interface MicroAgentSpec {
  endpoint: DiscoveredEndpoint;    // from Phase 1A crawl
  vulnClass: ExposureType;         // sqli | xss | ssrf | auth_bypass | ...
  depth: number;                   // 0 = primary target, 1+ = subdomain
  chainId: string;
  targetUrl: string;
}

export interface MicroAgentResult {
  spec: MicroAgentSpec;
  finding: BreachFinding | null;   // null = clean / no vuln
  evidence: ExploitEvidence[];
  credentials: HarvestedCredential[];
  durationMs: number;
  agentId: string;
}

export class MicroAgentOrchestrator {
  private semaphore: Semaphore;
  private results: MicroAgentResult[] = [];

  constructor(private maxConcurrent: number = 50) {
    this.semaphore = new Semaphore(maxConcurrent);
  }

  // Fan out: endpoints × vulnClasses → agent specs
  buildAgentSpecs(
    endpoints: DiscoveredEndpoint[],
    scope: ExposureType[],
    chainId: string,
```

```
    targetUrl: string,
  ): MicroAgentSpec[] {
    const specs: MicroAgentSpec[] = [];
    for (const endpoint of endpoints) {
      for (const vulnClass of scope) {
        // Skip irrelevant combinations early
        if (!isVulnClassApplicable(endpoint, vulnClass)) continue;
        specs.push({ endpoint, vulnClass, depth: 0, chainId, targetUrl });
      }
    }
    return specs;
  }

  // Execute all specs with semaphore-controlled concurrency
  async dispatch(
    specs: MicroAgentSpec[],
    onProgress: (completed: number, total: number, finding: MicroAgentResult) =>
void
  ): Promise<MicroAgentResult[]> {
    const results: MicroAgentResult[] = [];
    let completed = 0;

    await Promise.all(specs.map(async (spec) => {
      await this.semaphore.acquire();
      try {
        const result = await this.runMicroAgent(spec);
        results.push(result);
        onProgress(++completed, specs.length, result);
      } finally {
        this.semaphore.release();
      }
    }));

    return results;
  }

  private async runMicroAgent(spec: MicroAgentSpec): Promise<MicroAgentResult> {
    const agentId = `micro-${spec.vulnClass}-${randomUUID().slice(0, 6)}`;
    const start = Date.now();

    // 1. Fire targeted payloads for this specific vuln class against this endpoint
    const payloads = generatePayloadsForVulnClass(spec.vulnClass, spec.endpoint);
    const attempts = await firePayloadBatch(payloads, spec.endpoint, { timeout: 6000
});

    // 2. Validate responses with a focused LLM call (single focused prompt, not
full pipeline)
    const validated = await validateAttempts(attempts, spec.vulnClass);

    // 3. Extract any credentials from responses
    const credentials = extractCredentials(validated);
```

```
    return {
      spec,
      finding: validated.length > 0 ? buildFinding(validated, spec) : null,
      evidence: validated.map(a => a.evidence).flat(),
      credentials,
      durationMs: Date.now() - start,
      agentId,
    };
  }
}
```

Semaphore Implementation

```
// server/lib/semaphore.ts
export class Semaphore {
  private queue: (() => void)[] = [];
  private count: number;

  constructor(private max: number) {
    this.count = max;
  }

  async acquire(): Promise<void> {
    if (this.count > 0) {
      this.count--;
      return;
    }
    await new Promise<void>(resolve => this.queue.push(resolve));
  }

  release(): void {
    if (this.queue.length > 0) {
      this.queue.shift()!();
    } else {
      this.count++;
    }
  }
}
```

Wiring Into breach-orchestrator.ts — Phase 1B Replacement

Replace the current sequential `for (const exposureType of [...])` loop in `executeApplicationCompromise()`:

```
// BEFORE (sequential, 3 calls)
for (const exposureType of ["app_logic", "cve", "api_sequence_abuse"]) {
  const result = await runAgentOrchestrator(assetId, exposureType, ...)
}
```

```
// AFTER (parallel micro-agents)
const orchestrator = new MicroAgentOrchestrator(MAX_CONCURRENT_AGENTS);

// Build agent specs from Phase 1A crawl results
const agentSpecs = orchestrator.buildAgentSpecs(
  activeExploitResult.endpoints,       // crawled endpoints from Phase 1A
  exploitTarget.scope.exposureTypes,   // configured vuln classes
  chain.id,
  targetUrl,
);

onProgress(chain.id, "application_compromise", 50,
  `Dispatching ${agentSpecs.length} parallel micro-agents ` +
  `(${activeExploitResult.endpoints.length} endpoints × ` +
  `${exploitTarget.scope.exposureTypes.length} vuln classes)...`
);

const microResults = await orchestrator.dispatch(
  agentSpecs,
  (completed, total, result) => {
    onProgress(chain.id, "application_compromise",
      50 + Math.round((completed / total) * 40),
      `[${completed}/${total}] ${result.spec.vulnClass} @
${result.spec.endpoint.url}` +
      (result.finding ? ` → ${result.finding.severity} finding` : "")
    );
  }
);

// Merge all micro-agent findings into phase output
for (const mr of microResults) {
  if (mr.finding) findings.push(mr.finding);
  for (const cred of mr.credentials) {
    if (!newCredentials.find(c => c.valueHash === cred.hash)) {
      newCredentials.push(breachCredFromHarvested(cred));
    }
  }
}
```

Scale Math

Target Size	Endpoints	Vuln Classes	Agents	Wall-Clock (50 concurrent)
Small app	15	8	120	~30s
Medium app (Juice Shop)	30	10	300	~60s
Large enterprise target	80	10	800	~3–5 min
With subdomains (×5)	80 × 5	10	4,000	~15 min

Each agent makes 2–5 HTTP requests + 1 focused LLM validation call. At 50 concurrent agents with 6s timeout each, a 300-agent scan runs in ~36 seconds of wall-clock time.

Applicability Filter

Not every vuln class applies to every endpoint. Adding an early filter before agent dispatch prevents 30-40% of unnecessary agents:

```
function isVulnClassApplicable(
  endpoint: DiscoveredEndpoint,
  vulnClass: ExposureType
): boolean {
  const hasParams = endpoint.parameters.length > 0;
  const isPost = endpoint.method === "POST" || endpoint.method === "PUT";
  const hasAuth = endpoint.authenticated;

  switch (vulnClass) {
    case "sqli":         return hasParams;
    case "xss":          return hasParams;
    case "ssrf":         return hasParams && endpoint.parameters.some(p => p.type
=== "string");
    case "auth_bypass":  return hasAuth;
    case "idor":         return hasParams && endpoint.parameters.some(p =>
/id|uid|user/i.test(p.name));
    case "path_traversal": return hasParams && endpoint.parameters.some(p =>
/path|file|dir/i.test(p.name));
    case "command_injection": return isPost && hasParams;
    case "jwt_abuse":    return hasAuth &&
endpoint.headers["authorization"]?.includes("Bearer");
    case "api_abuse":    return true;
    case "business_logic": return isPost;
    default:             return true;
  }
}
```

Timeline

Week	Deliverable
Week 1	`Semaphore` + `MicroAgentOrchestrator` core class
Week 1	Per-vuln-class focused prompt templates (replace generic pipeline calls)
Week 2	Wire into `executeApplicationCompromise()` replacing sequential loop
Week 2	Applicability filter + agent spec builder
Week 3	Load testing: 300+ agents against Juice Shop, validate no false positive regression
Week 3	`subAgentRuns` UI extended to show per-agent status in breach chain view

---

CAPABILITY 2: CODE-TO-RUNTIME CONTEXT

Current State

The Go agent (`odinforge-agent/internal/collector/`) already collects, per enrolled host:

- **Open ports** — `GetOpenPorts()` → `PortInfo{Port, Protocol, Process, PID, Address}`
- **Running services** — `GetRunningServices()` → process list with names and PIDs
- **Network interfaces** — `GetNetworkInfo()` → `NetworkInfo{Interfaces[], PrimaryIP}`
- **System info** — OS, hostname, container runtime detection

This telemetry is queued and sent to OdinForge, but today it is used only for the agent management dashboard. It is never pulled back into any breach chain phase as context.

What Closing This Gap Gives You

Aikido's moat is seeing source code + dependencies + runtime simultaneously. OdinForge can get 60-70% of the value without source code access by injecting host telemetry into the LLM prompts that drive Phase 3 (Cloud IAM), Phase 4 (K8s Breakout), and Phase 5 (Lateral Movement). Concretely:

- **Phase 3 (Cloud IAM):** "The agent on host `api-prod-01` sees process `aws-iam-authenticator` running on PID 4812 and a file at `/home/ubuntu/.aws/credentials` referenced in process env." This radically changes what the IAM escalation LLM can reason about.
- **Phase 4 (K8s):** "Agent sees port 6443 listening locally (not exposed externally) and a service named `kube-apiserver`." Confirms K8s presence without a TCP probe from the outside.
- **Phase 5 (Lateral Movement):** "The agent on `db-server-02` sees port 5432 listening on `0.0.0.0` with process `postgres`. The agent on `web-server-01` has a file `/var/www/html/.env` with environment variables including `DATABASE_URL`." This is chain-of-custody evidence for a real credential → real database pivot.

New File: `server/services/aev/runtime-context-broker.ts`

```
import { storage } from "../../storage";
import type { BreachPhaseContext } from "@shared/schema";

export interface HostRuntimeContext {
  assetId: string;
  agentId: string;
  collectedAt: string;
  openPorts: Array<{ port: number; protocol: string; process?: string; pid?: string
}>;
  runningServices: Array<{ name: string; pid: string; status: string }>;
  networkInterfaces: Array<{ name: string; ipv4Addrs: string[]; hwAddr: string }>;
  primaryIp: string;
  containerRuntime: string | null;   // "docker" | "containerd" | null
  osName: string;
  hostname: string;
  // Derived signals — computed from raw telemetry
  signals: RuntimeSignal[];
}
```

```
export interface RuntimeSignal {
  type: "open_db_port" | "cloud_credential_file" | "container_runtime" |
        "k8s_component" | "internal_service" | "env_file_detected";
  description: string;
  severity: "high" | "medium" | "low";
  data: Record<string, unknown>;
}

export class RuntimeContextBroker {
  // Pull the latest telemetry event for a given asset from storage,
  // parse it into a structured HostRuntimeContext, and derive signals.
  async getContextForAsset(assetId: string): Promise<HostRuntimeContext | null> {
    const agent = await storage.getAgentByAssetId(assetId);
    if (!agent) return null;

    const latestEvent = await storage.getLatestAgentTelemetry(agent.id);
    if (!latestEvent) return null;

    const payload = latestEvent.payload as any;
    const ports: HostRuntimeContext["openPorts"] = (payload.ports || []).map((p:
any) => ({
      port: p.port,
      protocol: p.protocol,
      process: p.process,
      pid: p.pid,
    }));

    const services: HostRuntimeContext["runningServices"] = (payload.services ||
[]).map((s: any) => ({
      name: s.name,
      pid: String(s.pid || ""),
      status: s.status || "running",
    }));

    const network = payload.network || {};
    const ifaces = (network.interfaces || []).map((i: any) => ({
      name: i.name,
      ipv4Addrs: i.ipv4_addrs || [],
      hwAddr: i.hw_addr || "",
    }));

    const context: HostRuntimeContext = {
      assetId,
      agentId: agent.id,
      collectedAt: latestEvent.createdAt?.toISOString() || new Date().toISOString(),
      openPorts: ports,
      runningServices: services,
      networkInterfaces: ifaces,
      primaryIp: network.primary_ip || "",
      containerRuntime: payload.container?.runtime || null,
      osName: payload.system?.os_name || "unknown",
```

```
      hostname: payload.system?.hostname || assetId,
      signals: [],
    };

    // Derive high-value signals from raw telemetry
    context.signals = this.deriveSignals(context);
    return context;
  }

  private deriveSignals(ctx: HostRuntimeContext): RuntimeSignal[] {
    const signals: RuntimeSignal[] = [];
    const DB_PORTS = [5432, 3306, 27017, 6379, 1433, 5984, 9200];
    const K8S_SERVICES = ["kube-apiserver", "kubelet", "etcd", "kube-scheduler"];
    const CLOUD_CRED_PROCESSES = ["aws-iam-authenticator", "aws-vault", "gcloud",
"az"];

    for (const p of ctx.openPorts) {
      if (DB_PORTS.includes(p.port)) {
        signals.push({
          type: "open_db_port",
          severity: "high",
          description: `Database port ${p.port} (${p.process || "unknown process"})
open on host. ` +
            `If credentials are available, direct database access may be possible.`,
          data: { port: p.port, process: p.process, pid: p.pid },
        });
      }
    }

    for (const svc of ctx.runningServices) {
      if (K8S_SERVICES.some(k => svc.name.includes(k))) {
        signals.push({
          type: "k8s_component",
          severity: "high",
          description: `Kubernetes component "${svc.name}" (PID ${svc.pid})
confirmed running. ` +
            `K8s API and cluster state may be accessible from this host.`,
          data: { service: svc.name, pid: svc.pid },
        });
      }
      if (CLOUD_CRED_PROCESSES.some(c => svc.name.includes(c))) {
        signals.push({
          type: "cloud_credential_file",
          severity: "high",
          description: `Cloud credential process "${svc.name}" detected. ` +
            `AWS/GCP/Azure credentials may be accessible via this host's process
environment.`,
          data: { service: svc.name },
        });
      }
    }
```

```
    if (ctx.containerRuntime) {
      signals.push({
        type: "container_runtime",
        severity: "medium",
        description: `Container runtime "${ctx.containerRuntime}" active. ` +
          `Container escape or privilege escalation via Docker socket may be
possible.`,
        data: { runtime: ctx.containerRuntime },
      });
    }

    return signals;
  }

  // Format context as a structured prompt block for LLM injection
  formatForPrompt(ctx: HostRuntimeContext): string {
    const lines: string[] = [
      `## RUNTIME CONTEXT — ${ctx.hostname} (${ctx.primaryIp})`,
      `Agent telemetry collected at: ${ctx.collectedAt}`,
      `OS: ${ctx.osName}${ctx.containerRuntime ? ` | Container Runtime:
${ctx.containerRuntime}` : ""}`,
      "",
      `### Open Ports (${ctx.openPorts.length})`,
      ...ctx.openPorts.slice(0, 20).map(p =>
        `- ${p.port}/${p.protocol}${p.process ? ` → process: ${p.process} (PID
${p.pid})` : ""}`
      ),
      "",
      `### Running Services (${ctx.runningServices.length})`,
      ...ctx.runningServices.slice(0, 15).map(s => `- ${s.name} (PID ${s.pid})`),
      "",
      `### High-Value Signals`,
      ...ctx.signals.map(s => `- [${s.severity.toUpperCase()}] ${s.description}`),
    ];
    return lines.join("\n");
  }
}

export const runtimeContextBroker = new RuntimeContextBroker();
```

Injection Points in breach-orchestrator.ts

Phase 3 — Cloud IAM Escalation:

```
// Before running IAM analysis, pull runtime context for all chain assets
const runtimeContexts = await Promise.all(
  (chain.assetIds as string[]).map(id =>
runtimeContextBroker.getContextForAsset(id))
);
const validContexts = runtimeContexts.filter(Boolean) as HostRuntimeContext[];
const runtimeBlock = validContexts.map(ctx =>
```

```
runtimeContextBroker.formatForPrompt(ctx)).join("\n\n");

// Inject into the LLM description for IAM evaluation
const contextDescription = [
  `Breach chain ${chain.id}: Cloud IAM Escalation phase.`,
  `Prior access: ${context.currentPrivilegeLevel}.`,
  runtimeBlock.length > 0 ? `\n\nHOST TELEMETRY FROM ENROLLED
AGENTS:\n${runtimeBlock}` : "",
].join(" ");
```

Phase 5 — Lateral Movement:

```
// Inject per-host runtime context into the PivotQueue seed
for (const asset of context.compromisedAssets) {
  const runtimeCtx = await runtimeContextBroker.getContextForAsset(asset.assetId);
  if (runtimeCtx) {
    // Attach runtime signals to the pivot queue item so the SubAgent sees them
    pivotQueue.enqueueWithContext(
      asset.name || asset.assetId,
      0,
      "phase1_compromise",
      runtimeCtx.signals
    );
  }
}
```

Go Agent Extension — Env File Detection

Add env file scanning to the Go agent collector so it surfaces `.env` files, config files with credential patterns, and AWS/GCP credential paths:

```
// odinforge-agent/internal/collector/config_files.go

package collector

import (
    "os"
    "path/filepath"
    "regexp"
    "strings"
)

type ConfigFileSignal struct {
    Path     string   `json:"path"`
    Type     string   `json:"type"`   // "env_file" | "aws_creds" | "ssh_key" |
"kubeconfig"
    Patterns []string `json:"patterns"` // matched patterns (redacted values)
}

var credentialPatterns = []*regexp.Regexp{
```

```go
    regexp.MustCompile(`(?i)(password|passwd|pwd)\s*=\s*\S+`),
    regexp.MustCompile(`(?i)(api_key|apikey|api-key)\s*=\s*\S+`),
    regexp.MustCompile(`(?i)(aws_access_key|aws_secret)\s*=\s*\S+`),
    regexp.MustCompile(`(?i)DATABASE_URL\s*=\s*\S+`),
    regexp.MustCompile(`(?i)(token|bearer|secret)\s*=\s*\S+`),
}

var highValuePaths = []string{
    ".env", ".env.local", ".env.production",
    "config/database.yml", "config/secrets.yml",
    ".aws/credentials", ".kube/config",
    "docker-compose.yml", "docker-compose.override.yml",
}

func GetConfigFileSignals() []ConfigFileSignal {
    var signals []ConfigFileSignal
    home, _ := os.UserHomeDir()
    searchRoots := []string{home, "/etc", "/var/www", "/opt", "/srv"}

    for _, root := range searchRoots {
        for _, target := range highValuePaths {
            fullPath := filepath.Join(root, target)
            if data, err := os.ReadFile(fullPath); err == nil {
                var matched []string
                for _, re := range credentialPatterns {
                    if m := re.FindString(string(data)); m != "" {
                        // Redact the value — report only the key name
                        key := strings.Split(m, "=")[0]
                        matched = append(matched, strings.TrimSpace(key)+"=***")
                    }
                }
                if len(matched) > 0 {
                    signals = append(signals, ConfigFileSignal{
                        Path:     fullPath,
                        Type:     classifyFile(target),
                        Patterns: matched,
                    })
                }
            }
        }
    }
    return signals
}

func classifyFile(name string) string {
    if strings.HasPrefix(name, ".env") { return "env_file" }
    if strings.Contains(name, ".aws") { return "aws_creds" }
    if strings.Contains(name, ".kube") { return "kubeconfig" }
    return "config_file"
}
```

Timeline

Week	Deliverable
Week 2	`RuntimeContextBroker` class + storage queries for agent telemetry
Week 2	Inject runtime block into Phase 3 IAM + Phase 5 Lateral LLM prompts
Week 3	Go agent env file scanner (`config_files.go`)
Week 3	Deploy agent update, validate telemetry shows `config_file_signals` in payload
Week 4	Wire config file signals into `deriveSignals()` and Phase 5 pivot context

---

CAPABILITY 3: AUTO-REMEDIATION LOOP

Current State

The verify endpoint already exists:

```
POST /api/aev/evaluations/:id/verify-fix
```

The Defender's Mirror (`server/services/defenders-mirror.ts`) generates detection rules (Sigma, YARA, Splunk SPL) per finding. But OdinForge stops at detection. There is no "generate fix → store fix → re-run → confirm closed" loop.

Target Flow

```
Confirmed Finding (PROVEN by EvidenceQualityGate)
    ↓
RemediationEngine.generateFix(finding)
    ↓
FixProposal { type: "code_patch" | "config_change" | "waf_rule", content, priority }
    ↓
POST /api/aev/evaluations/:id/verify-fix
    ↓
Re-run targeted micro-agent against same endpoint
    ↓
FixVerification { status: "confirmed_fixed" | "still_vulnerable" | "partially_fixed" }
    ↓
Store in breach chain result + surface in report
```

New File: `server/services/aev/remediation-engine.ts`

```typescr
import type { BreachPhaseResult } from "@shared/schema";
import { openai } from "../llm-client";

export type FixType = "code_patch" | "config_change" | "waf_rule" |
"dependency_update";

export interface FixProposal {
  id: string;
  findingId: string;
  type: FixType;
  priority: "immediate" | "high" | "medium";
  title: string;
  description: string;
  // Concrete fix artifacts
  codeSnippet?: string;     // if type === "code_patch"
  configChange?: string;    // if type === "config_change"
  wafRule?: string;         // if type === "waf_rule"
  // Estimated remediation effort
```

```
  effortEstimate: "< 1 hour" | "1–4 hours" | "1–2 days" | "> 2 days";
  // Verification method after fix is applied
  verificationPayload: string;  // the original payload that triggered the vuln
}

export interface FixVerification {
  proposalId: string;
  findingId: string;
  status: "confirmed_fixed" | "still_vulnerable" | "partially_fixed" |
"unable_to_verify";
  verifiedAt: string;
  evidence: string;
  retestPayload: string;
  retestResponse: string;
}

export class RemediationEngine {
  async generateFix(finding: {
    id: string;
    title: string;
    description: string;
    severity: string;
    technique?: string;
    mitreId?: string;
    evidence?: string;
    endpointUrl?: string;
    payload?: string;
  }): Promise<FixProposal> {
    const prompt = `You are a senior security engineer generating a concrete
remediation for a confirmed vulnerability.

Finding:
- Title: ${finding.title}
- Severity: ${finding.severity}
- Description: ${finding.description}
- MITRE ATT&CK: ${finding.mitreId || "N/A"}
- Endpoint: ${finding.endpointUrl || "N/A"}
- Triggering Payload: ${finding.payload || "N/A"}
- Evidence: ${finding.evidence || "N/A"}

Generate a JSON remediation proposal with this structure:
{
  "type": "code_patch" | "config_change" | "waf_rule" | "dependency_update",
  "priority": "immediate" | "high" | "medium",
  "title": "Short fix title",
  "description": "Plain-English explanation of the fix",
  "codeSnippet": "Code showing the fix (if applicable)",
  "configChange": "Config change (if applicable)",
  "wafRule": "WAF rule (if applicable)",
  "effortEstimate": "< 1 hour" | "1–4 hours" | "1–2 days" | "> 2 days",
  "verificationPayload": "The exact payload to re-test after the fix"
}
```

```
Be specific. No generic advice. Return only JSON.`;

    const response = await openai.chat.completions.create({
      model: "gpt-4o",
      messages: [{ role: "user", content: prompt }],
      response_format: { type: "json_object" },
      max_tokens: 1200,
    });

    const raw = JSON.parse(response.choices[0].message.content || "{}");
    return {
      id: `fix-${randomUUID().slice(0, 8)}`,
      findingId: finding.id,
      ...raw,
    };
  }

  async verifyFix(
    proposal: FixProposal,
    targetUrl: string,
  ): Promise<FixVerification> {
    // Re-fire the original triggering payload against the endpoint after fix is
applied
    const controller = new AbortController();
    const timeout = setTimeout(() => controller.abort(), 10000);

    try {
      const response = await fetch(targetUrl, {
        method: "POST",
        headers: { "Content-Type": "application/json", "User-Agent": "OdinForge-
AEV/Verify" },
        body: JSON.stringify({ payload: proposal.verificationPayload }),
        signal: controller.signal,
      });

      const body = await response.text().catch(() => "");

      // Check if the vulnerability indicators are still present
      const stillVulnerable = this.detectVulnIndicators(body, response.status,
proposal);

      return {
        proposalId: proposal.id,
        findingId: proposal.findingId,
        status: stillVulnerable ? "still_vulnerable" : "confirmed_fixed",
        verifiedAt: new Date().toISOString(),
        evidence: `HTTP ${response.status} — ${body.substring(0, 500)}`,
        retestPayload: proposal.verificationPayload,
        retestResponse: body.substring(0, 1000),
      };
    } catch (err: any) {
```

```
      return {
        proposalId: proposal.id,
        findingId: proposal.findingId,
        status: "unable_to_verify",
        verifiedAt: new Date().toISOString(),
        evidence: `Verification request failed: ${err.message}`,
        retestPayload: proposal.verificationPayload,
        retestResponse: "",
      };
    } finally {
      clearTimeout(timeout);
    }
  }

  private detectVulnIndicators(
    body: string,
    status: number,
    proposal: FixProposal
  ): boolean {
    // SQL injection indicators
    if (body.match(/SQL|syntax error|mysql_fetch|ORA-\d{5}|pg_query/i)) return true;
    // XSS reflection
    if (body.includes("<script>") || body.includes("onerror=")) return true;
    // SSRF indicators
    if (body.match(/internal|169\.254|10\.\d+|192\.168/)) return true;
    // Error disclosure
    if (status === 500 && body.length > 500) return true;
    return false;
  }
}

export const remediationEngine = new RemediationEngine();
```

API Endpoint — Generate Fix

```
// In server/routes.ts — extend existing verify-fix endpoint

app.post("/api/aev/evaluations/:id/generate-fix",
  apiRateLimiter, uiAuthMiddleware, requirePermission("evaluations:write"),
  async (req, res) => {
    const { findingId, finding } = req.body;
    if (!findingId || !finding) {
      return res.status(400).json({ error: "findingId and finding required" });
    }

    const proposal = await remediationEngine.generateFix(finding);
    // Store proposal against the evaluation
    await storage.storeFixProposal(req.params.id, proposal);
    res.json(proposal);
  }
```

```
);

// Extend existing verify-fix endpoint to use stored proposal
app.post("/api/aev/evaluations/:id/verify-fix",
  apiRateLimiter, uiAuthMiddleware, requirePermission("evaluations:write"),
  async (req, res) => {
    const { proposalId, targetUrl } = req.body;
    const proposal = await storage.getFixProposal(proposalId);
    if (!proposal) return res.status(404).json({ error: "Proposal not found" });

    const verification = await remediationEngine.verifyFix(proposal, targetUrl);
    await storage.storeFixVerification(req.params.id, verification);
    res.json(verification);
  }
);
```

Integration With Breach Chain

After the breach chain completes, for all `PROVEN` findings, auto-generate fix proposals as a background task:

```
// At the end of runBreachChain(), after final storage.updateBreachChain()
generateFixProposalsForChain(chainId, finalQualityVerdict.passed).catch(err => {
  console.error(`[BreachOrchestrator] Fix proposal generation failed for
${chainId}:`, err);
});

async function generateFixProposalsForChain(
  chainId: string,
  provenFindings: EvaluatedFinding[]
): Promise<void> {
  const criticalAndHigh = provenFindings.filter(
    f => f.finding.severity === "critical" || f.finding.severity === "high"
  );
  for (const ef of criticalAndHigh.slice(0, 20)) { // cap at 20 proposals per chain
    const proposal = await remediationEngine.generateFix({
      id: ef.finding.id,
      title: ef.finding.title,
      description: ef.finding.description,
      severity: ef.finding.severity,
      technique: ef.finding.technique,
      mitreId: ef.finding.mitreId,
    });
    await storage.storeFixProposal(chainId, proposal);
  }
}
```

Timeline

Week	Deliverable

Week 3	`RemediationEngine` class with `generateFix()`
Week 3	`POST /api/aev/evaluations/:id/generate-fix` endpoint
Week 4	Extend `verify-fix` to use stored proposals
Week 4	Background fix generation after breach chain completion
Week 4	Fix proposal display in report generator + breach chain detail view

---

COMBINED IMPLEMENTATION TIMELINE

8-Week Execution Plan

Weeks 1–2: Parallel Agents (Phase 1 Priority)

- `Semaphore` utility class
- `MicroAgentOrchestrator` with dispatch + progress reporting
- Per-vuln-class focused prompt templates
- Applicability filter (30-40% agent reduction)
- Wire into `executeApplicationCompromise()` — replace sequential loop
- Load test: 300+ agents against Juice Shop

Weeks 2–3: Runtime Context (Phase 2)

- `RuntimeContextBroker` — pull + parse agent telemetry by asset ID
- Signal derivation (DB ports, K8s services, cloud credential processes)
- Inject into Phase 3 (IAM) + Phase 5 (Lateral Movement) LLM prompts
- Go agent `config_files.go` — env file + credential path detection
- Deploy Go agent update, validate telemetry schema

Weeks 3–4: Auto-Remediation (Phase 3)

- `RemediationEngine.generateFix()` with GPT-4o structured output
- `verifyFix()` using original triggering payload re-test
- `POST /api/aev/evaluations/:id/generate-fix` endpoint
- Extend `verify-fix` endpoint to use stored proposals
- Background fix generation after breach chain completion

Weeks 5–6: Integration + Polish

- End-to-end test: full chain against Juice Shop with all three capabilities active
- `subAgentRuns` UI extended for micro-agent status visualization
- Fix proposals visible in breach chain report + detail view
- Runtime context visible in phase detail (what the agent saw vs. what was found)

Weeks 7–8: Performance + QA

- 50-agent concurrency load test (memory, rate limits, error rates)
- Runtime context injection regression test (no false positives from telemetry noise)
- Remediation loop closed-loop test: introduce a SQLi, generate fix, apply, verify
- Security test: verify micro-agents respect engagement scope, no out-of-scope requests

New Files Summary

File	Purpose
`server/lib/semaphore.ts`	Concurrency control primitive

`server/services/aev/micro-agent-orchestrator.ts`	Parallel agent fan-out engine
`server/services/aev/runtime-context-broker.ts`	Agent telemetry → LLM context
`server/services/aev/remediation-engine.ts`	Fix generation + verification
`odinforge-agent/internal/collector/config_files.go`	Go agent env/config file scanner

Modified Files Summary

File	Change
`server/services/breach-orchestrator.ts`	Replace sequential Phase 1B loop with MicroAgentOrchestrator; inject runtime context in Phase 3 + 5
`server/services/aev/pivot-queue.ts`	Add `enqueueWithContext()` for runtime signal attachment
`server/routes.ts`	Add `generate-fix` endpoint; extend `verify-fix`
`server/storage.ts`	Add `storeFixProposal()`, `getFixProposal()`, `storeFixVerification()`, `getAgentByAssetId()`, `getLatestAgentTelemetry()`

COMPETITIVE POSITIONING AFTER BUILD

Capability	OdinForge Today	OdinForge After Build	Aikido
Agent concurrency	1 engine + 3 sequential	50–300 concurrent micro-agents	100+ agents
Coverage per target	~3% of attack surface	60–90% of attack surface	~80%
Host-level context	None	Runtime telemetry via Go agent	Source code + SAST
Remediation	Detection rules only	Generate fix + verify closed	Apply patch + re-test
Time-to-results	30–90 seconds	3–8 minutes (deep scan)	5–15 minutes

The honest assessment: these three capabilities put OdinForge structurally competitive with a company that raised significant funding. Item 2 (runtime context) does not close the source-code gap — that remains Aikido's deepest moat — but the Go agent's host telemetry closes enough of it to produce qualitatively different findings. Item 1 (parallel agents) is the single highest-leverage change: it transforms OdinForge from a probe into a siege.